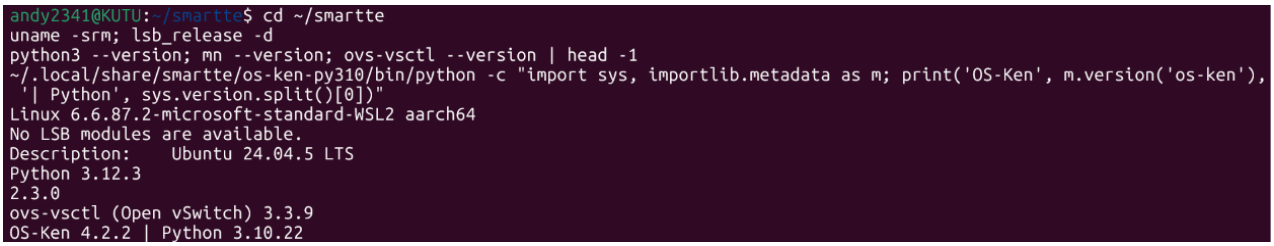
SmartTE: Screenshot Evidence

Centralized Traffic Engineering in Data Center Networks (Topic 9, BACSE203). Environment: WSL2 Ubuntu 24.04.5 (aarch64), Mininet 2.3.0, Open vSwitch 3.3.9, OS-Ken 4.2.2.

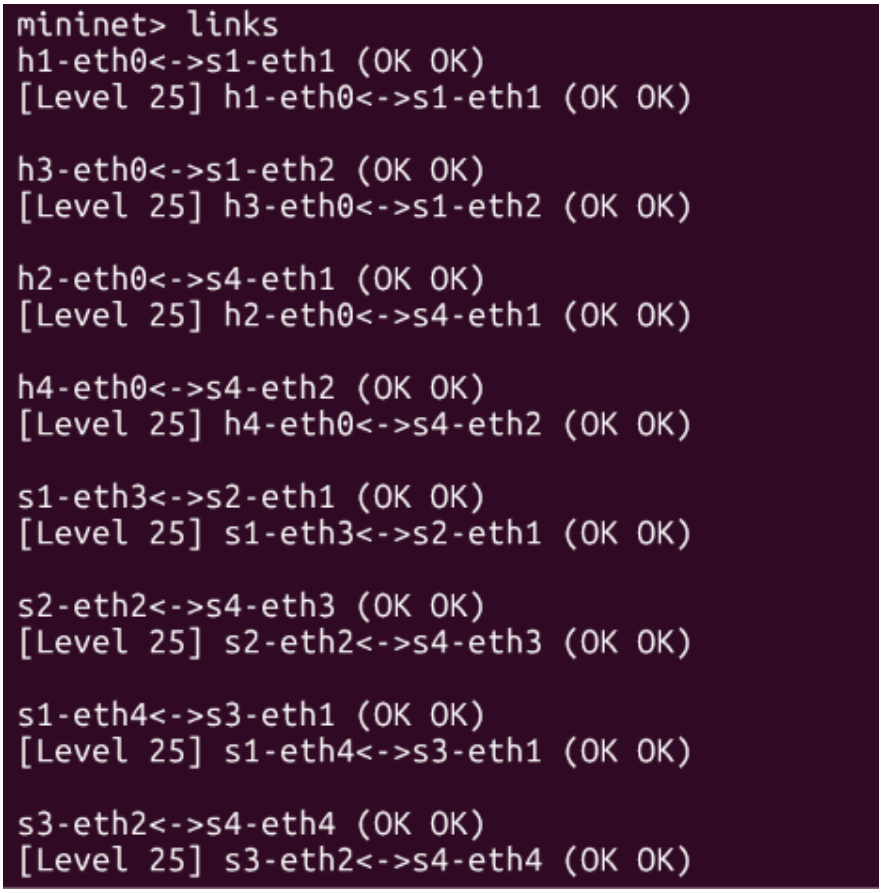
Provenance note. Figures 3 to 8 show saved results from the primary campaign `m5_campaign_20261011T115819Z` (110 rows, n=5 per cell, no significance test). They are archived evidence, not live runs. The secondary diagnostic campaign is not used. Figures 1, 2 and 9 are live terminal captures.

Figure 1. Software environment



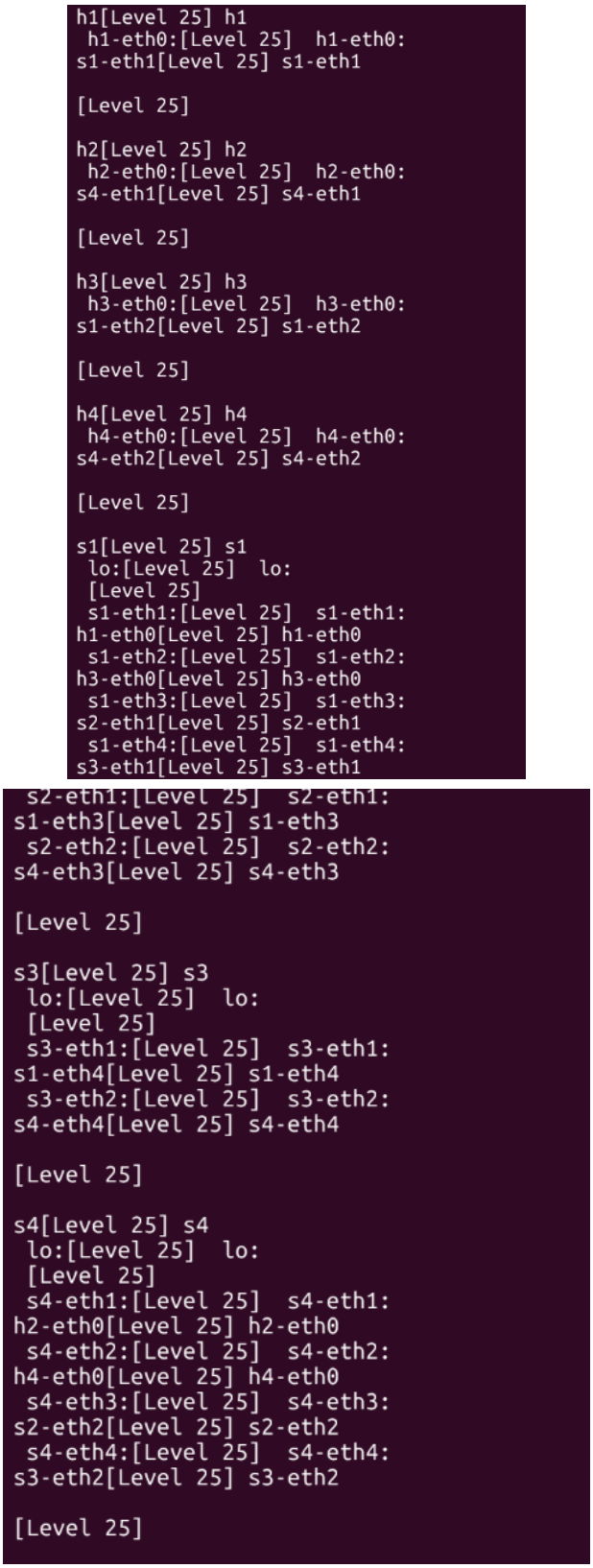
Versions of the stack: Linux 6.6.87.2 WSL2 aarch64, Ubuntu 24.04.5, Python 3.12.3, Mininet 2.3.0, Open vSwitch 3.3.9, and OS-Ken 4.2.2 running under Python 3.10.22 (isolated environment).

Figure 2. Topology links, static forwarding mode



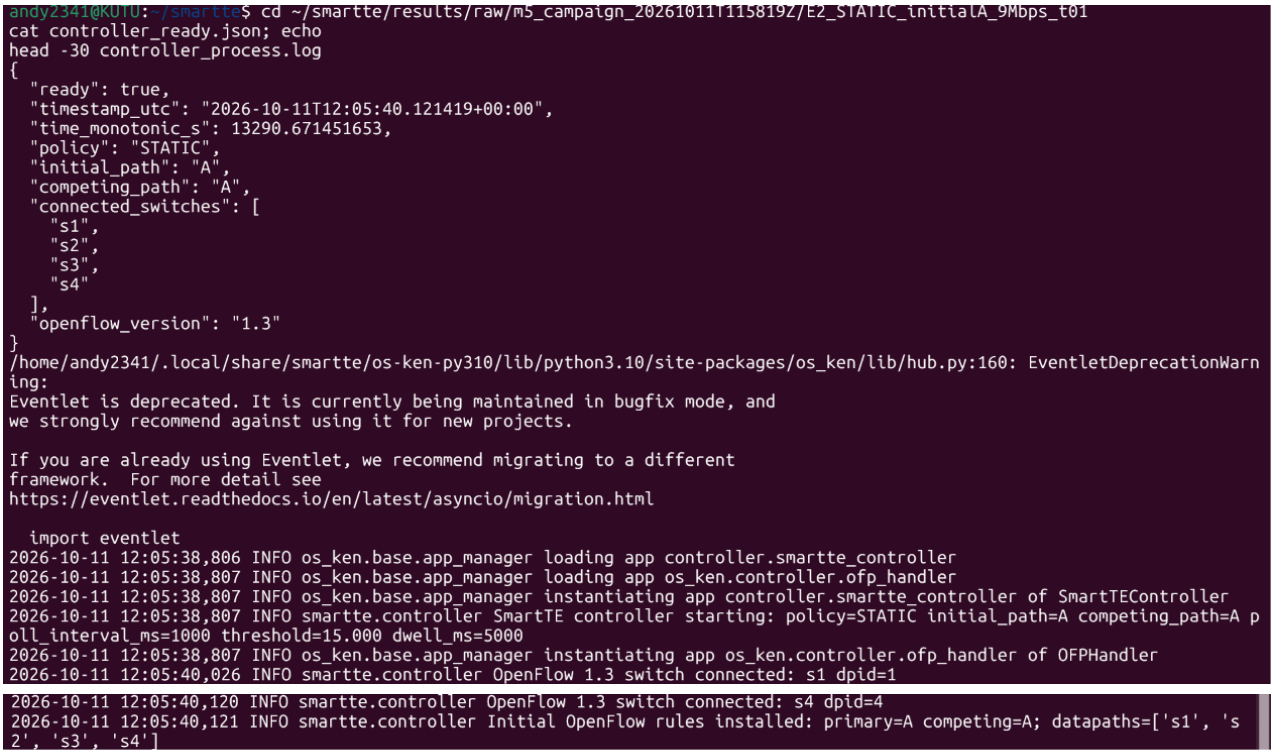
Mininet `links` output for the SmartTE diamond (launched with `--path A --competing A`, no OS-Ken controller): access links h1/h3-s1 and h2/h4-s4, Path A (s1-eth3/s2-eth1, s2-eth2/s4-eth3) and Path B (s1-eth4/s3-eth1, s3-eth2/s4-eth4). All eight links report OK. The duplicated lines come from Mininet's logging echo.

Figure 3. Topology node and port listing (net)



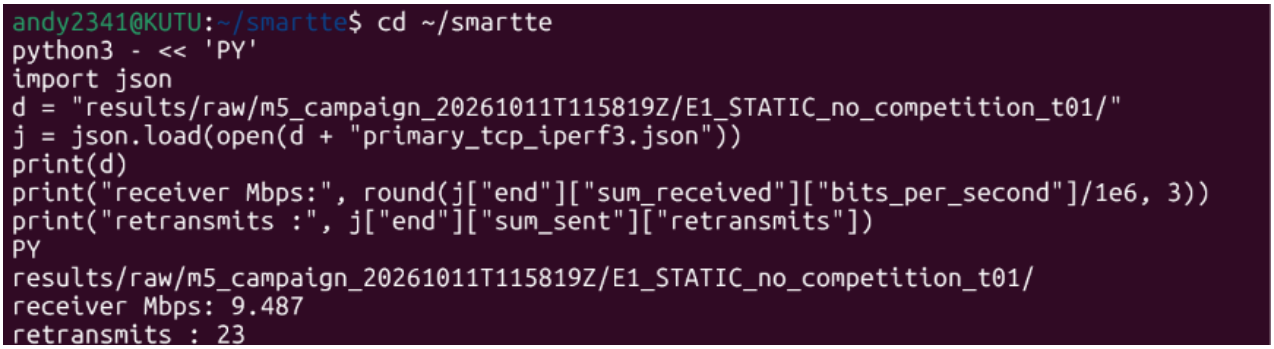
Mininet `net` output in two parts, showing four hosts, four switches and the port numbering used by the controller and flow rules (s1: eth1=h1, eth2=h3, eth3=s2, eth4=s3).

Figure 4. Controller startup record (archived, trial E2_STATIC_initialA_9Mbps_t01)



OS-Ken controller ready record: policy STATIC, OpenFlow 1.3, switches s1 to s4 connected, followed by the log lines showing switch connection and the initial OpenFlow rules installed on all four datapaths. Saved output from a campaign trial, not a live window.

Figure 5. Static baseline iperf3 result (archived)



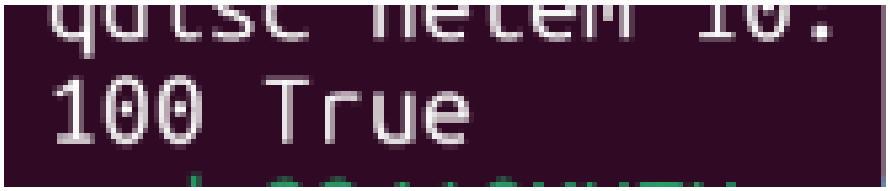
E1 static, no competing load: receiver-side TCP throughput 9.487 Mbps over a 10 Mbps core path, 23 retransmits (from `primary_tcp_iperf3.json`).

Figure 6. Competing traffic versus h1-h2 throughput (archived)

```
andy2341@KUTU:~/smartte$ cd ~/smartte
python3 - << 'PY'
import csv
rows = csv.DictReader(open("results/raw/m5_campaign_20261011T115819Z.csv"))
print("scenario | trial | offered_load_mbps | competing_achieved_mbps | tcp_throughput_mbps")
for r in rows:
    if r["scenario"].startswith("E2_STATIC"):
        print(r["scenario"], r["trial"], r["offered_load_mbps"], r["competing_achieved_mbps"], r["tcp_throughput_mbps"], sep
=" | ")
PY
scenario | trial | offered_load_mbps | competing_achieved_mbps | tcp_throughput_mbps
E2_STATIC_initialA_5Mbps | 1 | 5 | 4.942 | 5.776
E2_STATIC_initialB_5Mbps | 1 | 5 | 4.992 | 9.418
E2_STATIC_initialA_9Mbps | 1 | 9 | 8.618 | 3.175
E2_STATIC_initialB_9Mbps | 1 | 9 | 8.985 | 9.457
E2_STATIC_initialB_5Mbps | 2 | 5 | 4.992 | 9.403
E2_STATIC_initialA_9Mbps | 2 | 9 | 8.594 | 3.488
E2_STATIC_initialB_9Mbps | 2 | 9 | 8.985 | 9.464
E2_STATIC_initialA_5Mbps | 2 | 5 | 4.927 | 5.976
E2_STATIC_initialA_9Mbps | 3 | 9 | 8.731 | 3.232
E2_STATIC_initialB_9Mbps | 3 | 9 | 8.985 | 9.515
E2_STATIC_initialA_5Mbps | 3 | 5 | 4.956 | 5.981
E2_STATIC_initialB_5Mbps | 3 | 5 | 4.992 | 9.511
E2_STATIC_initialA_9Mbps | 4 | 9 | 8.614 | 3.331
E2_STATIC_initialB_9Mbps | 4 | 9 | 8.985 | 9.463
E2_STATIC_initialA_5Mbps | 4 | 5 | 4.924 | 6.012
E2_STATIC_initialB_5Mbps | 4 | 5 | 4.992 | 9.483
E2_STATIC_initialA_9Mbps | 5 | 9 | 8.621 | 3.331
E2_STATIC_initialB_9Mbps | 5 | 9 | 8.985 | 9.509
E2_STATIC_initialA_5Mbps | 5 | 5 | 4.93 | 5.983
E2_STATIC_initialB_5Mbps | 5 | 5 | 4.992 | 9.509
```

E2 static trials. With h1-h2 on Path A, the competing UDP flow (requested 5 and 9 Mbps; achieved about 4.9 and 8.6 Mbps) reduces h1-h2 TCP throughput to about 5.8 to 6.0 and 3.2 to 3.5 Mbps. With h1-h2 on Path B, throughput stays about 9.4 to 9.5 Mbps.

Figure 7. Queue verification (archived), cropped capture



Crop of the saved `tc` output and manifest check: a netem qdisc line and `100 True` (queue_limit_pkts = 100, queue_limit_verified = True). **Limitation:** this capture is cropped and does not show every interface. The full per-interface `limit 100` lines are in the trial's `*_tc.txt` file and manifest.json.

Figure 8. Unit tests

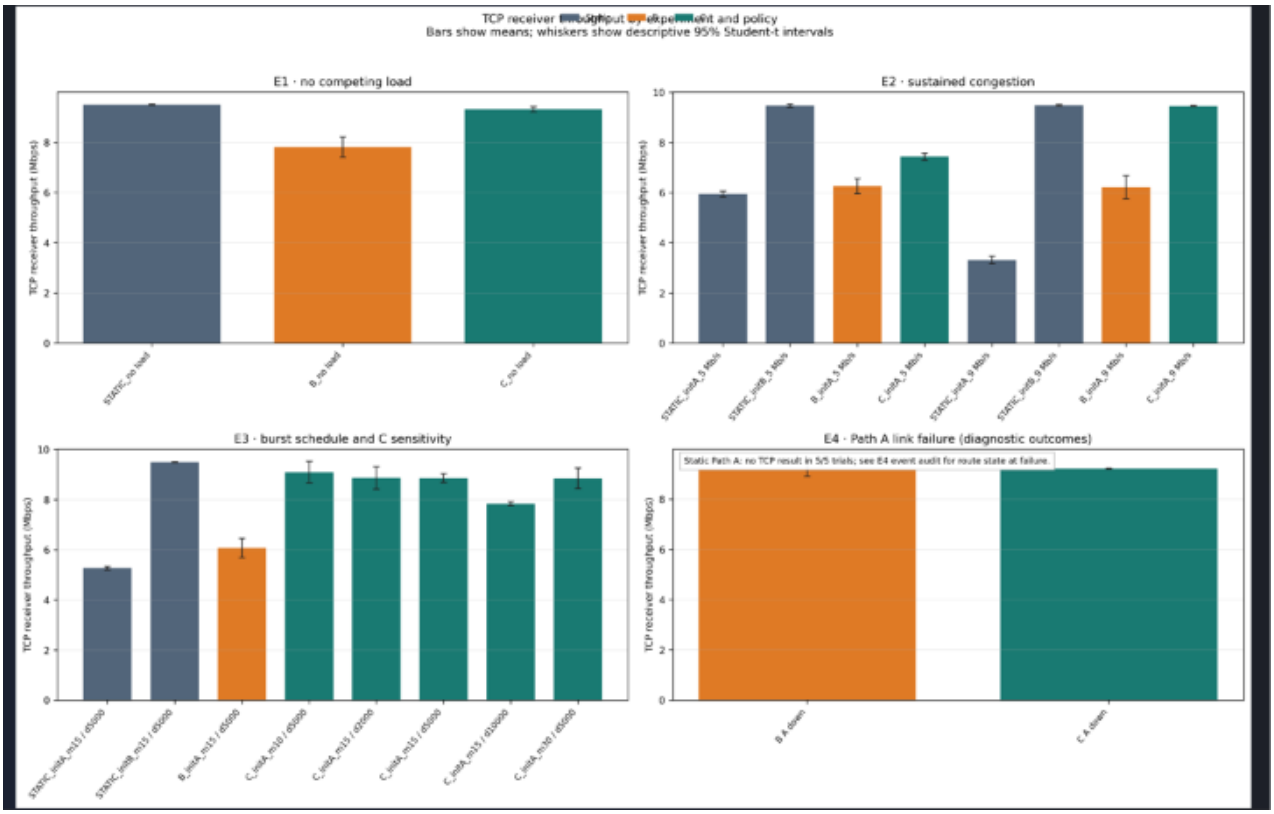
```
andy2341@KUTU:~/smartte$ cd ~/smartte
python3 -m unittest discover tests -v 2>&1 | tail -30
test_dwell_blocks_then_allows_a_qualified_switch (test_congestion_policy.HysteresisAndDwellTests.test_dwell_blocks_then_allows_a_qualified_switch) ... ok
test_margin_boundary_switches_and_below_boundary_holds (test_congestion_policy.HysteresisAndDwellTests.test_margin_boundary_switches_and_below_boundary_hold
s) ... ok
test_deterministic_tie_break_prefers_path_a (test_congestion_policy.PathScoringTests.test_deterministic_tie_break_prefers_path_a) ... ok
test_invalid_path_and_missing_scores_are_rejected (test_congestion_policy.PathScoringTests.test_invalid_path_and_missing_scores_are_rejected) ... ok
test_path_score_is_maximum_core_link_utilization (test_congestion_policy.PathScoringTests.test_path_score_is_maximum_core_link_utilization) ... ok
test_policy_b_switches_to_a_lower_score (test_congestion_policy.PathScoringTests.test_policy_b_switches_to_a_lower_score) ... ok
test_successful_path_count_ignores_rejected_switch_attempts (test_congestion_policy.PathScoringTests.test_successful_path_count_ignores_rejected_switch_atte
mpts) ... ok
test_unavailable_path_is_excluded_from_path_scores (test_congestion_policy.PathScoringTests.test_unavailable_path_is_excluded_from_path_scores) ... ok
test_counter_wrap_and_short_interval_are_not_measurements (test_congestion_policy.PortCounterTests.test_counter_wrap_and_short_interval_are_not_measurements
) ... ok
test_utilization_uses_byte_delta_interval_and_capacity (test_congestion_policy.PortCounterTests.test_utilization_uses_byte_delta_interval_and_capacity) ...
ok
test_waits_until_background_iperf_json_is_complete (test_run_static_baseline_helpers.BackgroundIperfOutputTests.test_waits_until_background_iperf_json_is_co
mplete) ... ok
test_h3_h4_duplicate_reply_fails_path_validation (test_run_static_baseline_helpers.PathEvidenceTests.test_h3_h4_duplicate_reply_fails_path_validation) ... o
k
test_h3_h4_path_a_passes_when_only_path_a_carries_packets (test_run_static_baseline_helpers.PathEvidenceTests.test_h3_h4_path_a_passes_when_only_path_a_carr
ies_packets) ... ok
test_h3_h4_without_rules_passes_only_when_both_paths_are_idle (test_run_static_baseline_helpers.PathEvidenceTests.test_h3_h4_without_rules_passes_only_when_
both_paths_are_idle) ... ok
test_packet_delta_sum_uses_colon_delimited_counter_keys (test_run_static_baseline_helpers.PathEvidenceTests.test_packet_delta_sum_uses_colon_delimited_count
er_keys) ... ok
test_parses_named_ports_with_rx_and_tx_on_separate_lines (test_run_static_baseline_helpers.PortCounterParserTests.test_parses_named_ports_with_rx_and_tx_on_
separate_lines) ... ok
test_parses_numeric_port_id_and_same_line_counters (test_run_static_baseline_helpers.PortCounterParserTests.test_parses_numeric_port_id_and_same_line_counte
rs) ... ok
test_rejects_inconsistent_default_limits (test_run_static_baseline_helpers.QueueLimitResolutionTests.test_rejects_inconsistent_default_limits) ... ok
test_resolves_mininet_default_from_tc_output (test_run_static_baseline_helpers.QueueLimitResolutionTests.test_resolves_mininet_default_from_tc_output) ... o
k
test_verifies_explicit_limit_on_every_interface (test_run_static_baseline_helpers.QueueLimitResolutionTests.test_verifies_explicit_limit_on_every_interface)
 ... ok

----------------------------------------------------------------------
Ran 20 tests in 0.505s

OK
```

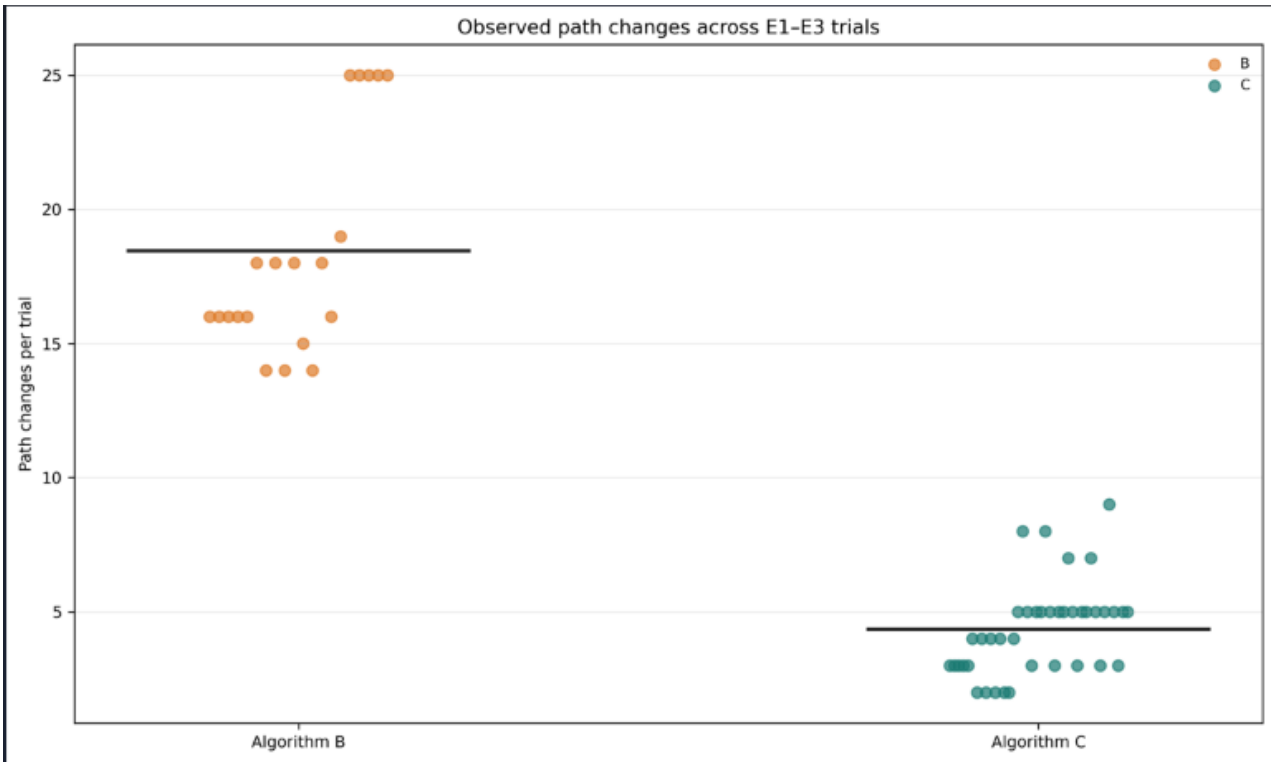
`python3 -m unittest discover tests -v`: 20 tests covering path scoring, hysteresis and dwell logic, counter-based utilisation, port-counter parsing and queue-limit resolution. Result: Ran 20 tests, OK.

Figure 9. Controller comparison: throughput by scenario (archived plot)



TCP receiver throughput by scenario and policy, with descriptive 95% Student-t intervals (n=5 per cell). Note: the legend overlaps the title text, and the E4 panel shows B and C only; static Path A had no TCP result in 5/5 trials (expected outage), as the in-plot annotation says. E4 does not establish a valid controller recovery-time comparison.

Figure 10. Path changes per trial, B versus C (archived plot)



Observed path changes per trial pooled across E1 to E3. Algorithm B switches far more often than Algorithm C (frozen margin 15, dwell 5 s, 1 s polling), including with no competing load, where the controlled flow's own traffic drives the utilisation signal.

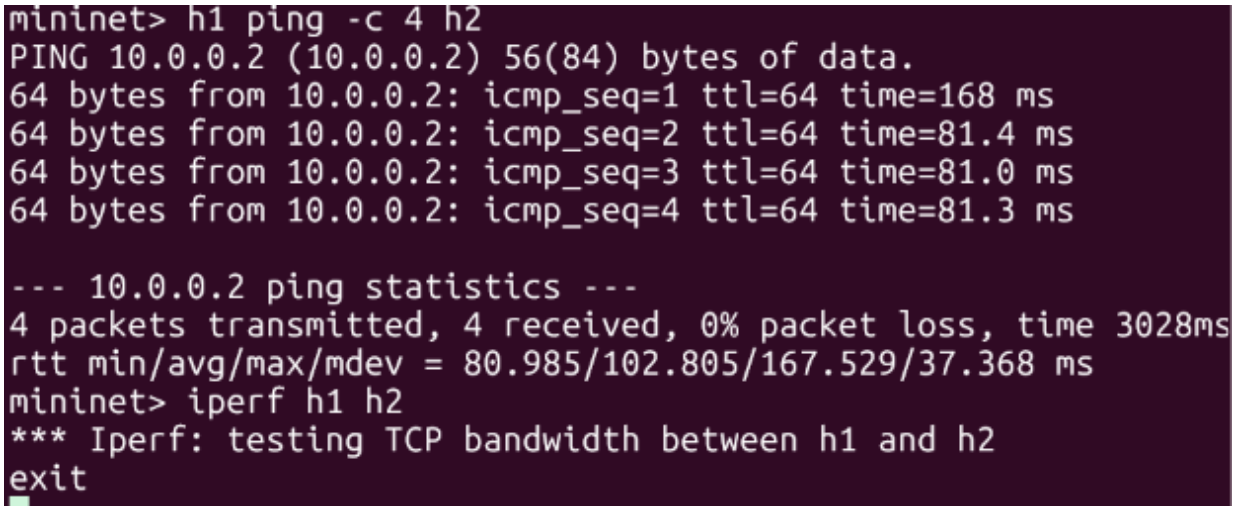
Figure 11. E4: raw path changes versus successful changes (archived)

alg trial	raw_path_changes	successful	attempts	rejected
static 1	0	0	0	0
b 1	17	1	17	16
c 1	11	1	11	10
b 2	17	1	17	16
c 2	12	1	12	11
static 2	0	0	0	0
c 3	12	1	12	11
static 3	0	0	0	0
b 3	17	1	17	16
static 4	0	0	0	0
b 4	17	1	17	16
c 4	12	1	12	11
b 5	17	1	17	16
c 5	12	1	12	11
static 5	0	0	0	0

Raw CSV path_changes counts switch decisions. Each B or C trial had exactly one successful path change; the remaining attempts were rejected after the link failed. Derived from controller event logs; the raw CSV is unchanged.

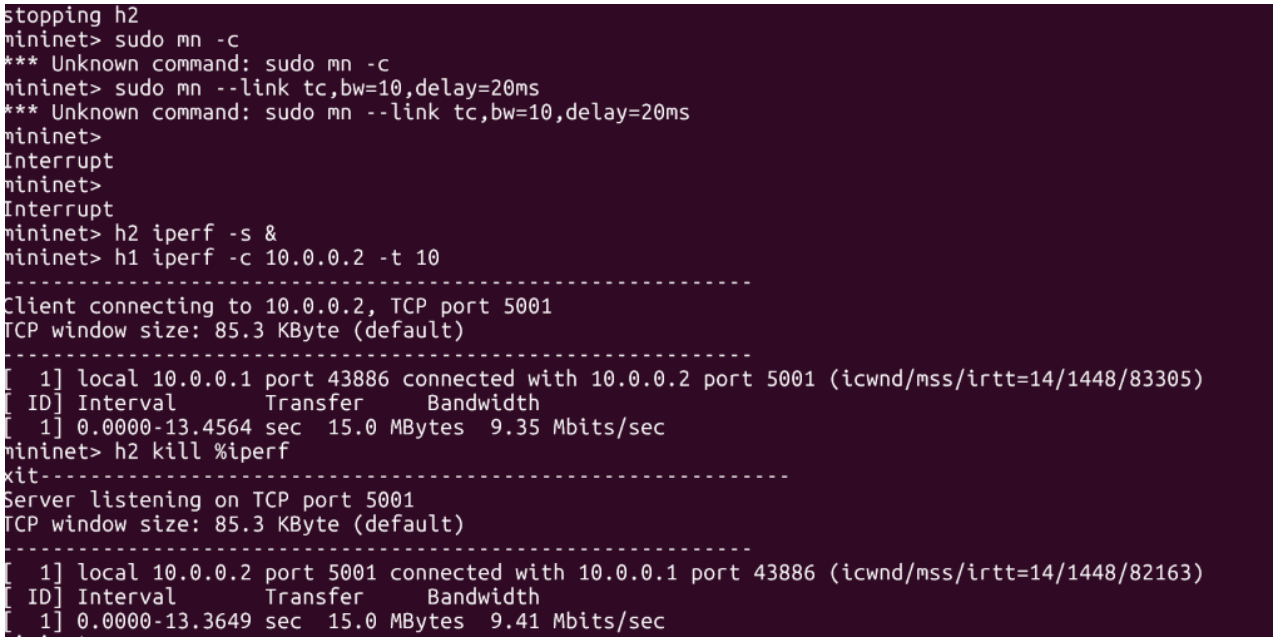
Appendix: environment validation (earlier milestone)

Figure A1. Delay emulation check



Default Mininet test with 10 Mbps / 20 ms links: steady-state RTT about 81 ms against 80 ms expected; first packet slower (ARP).

Figure A2. Manual iperf bandwidth check



Manual iperf over a 10 Mbps shaped link: 9.35 Mbit/s (client) and 9.41 Mbit/s (server). Mininet's built-in iperf command was not used because it hung. Lines showing 'Unknown command' were shell commands typed at the mininet prompt by mistake.